Rust for Secure and Sustainable Smart Metering

RustWeek 2026 - Industry Track

SCHUBERG PHILIS

Deciding to use Rust

- If you're here, you're weighing whether Rust is the right call for your organisation
- We had to make the same decision

  On a project that touches 8 million households
- I'll show you how we reasoned through it, how we carried it through the procurement, and share some of our experiences so far

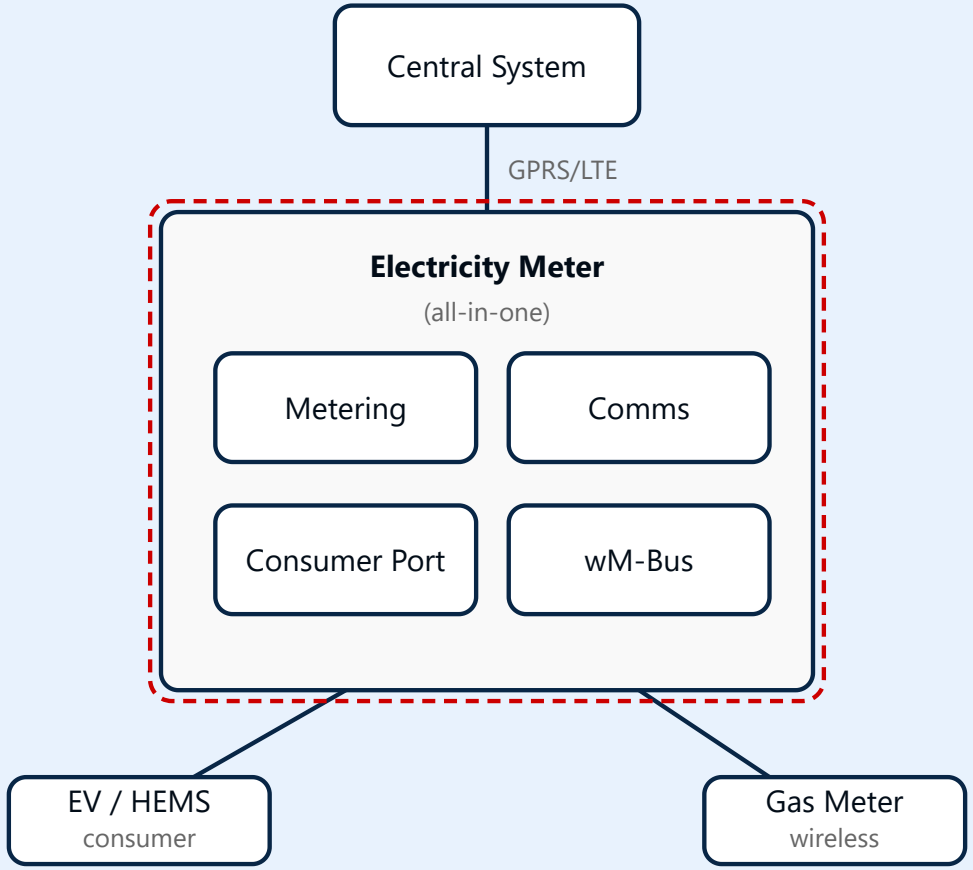
Who I Am

- **Ron van der Pols - Tech Lead at Schuberg Philis**
  Mission-critical IT - we run systems that cannot go down
- **Rust in production**
  Clients where we use Rust include NS (Dutch Railways, on the trains) and Heineken
- **Current project: NextGen**
  The DSOs' program developing DSMR6 — the next generation of smart meters in The Netherlands. We build the application layer.

Current Meters, gen 4 & 5

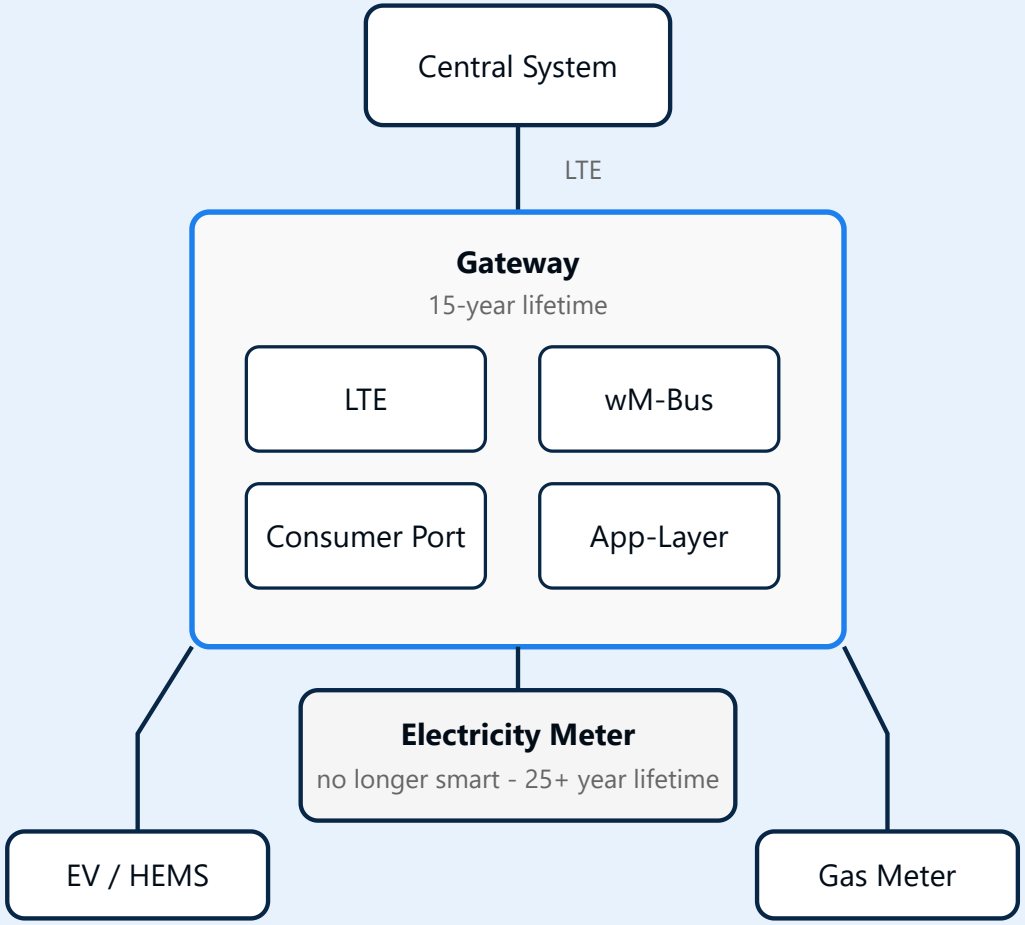
- All-in-one device
- Connected to gas meter
- Communicates with central system
- Consumer port

Problem: one component ages out → whole meter needs to be replaced



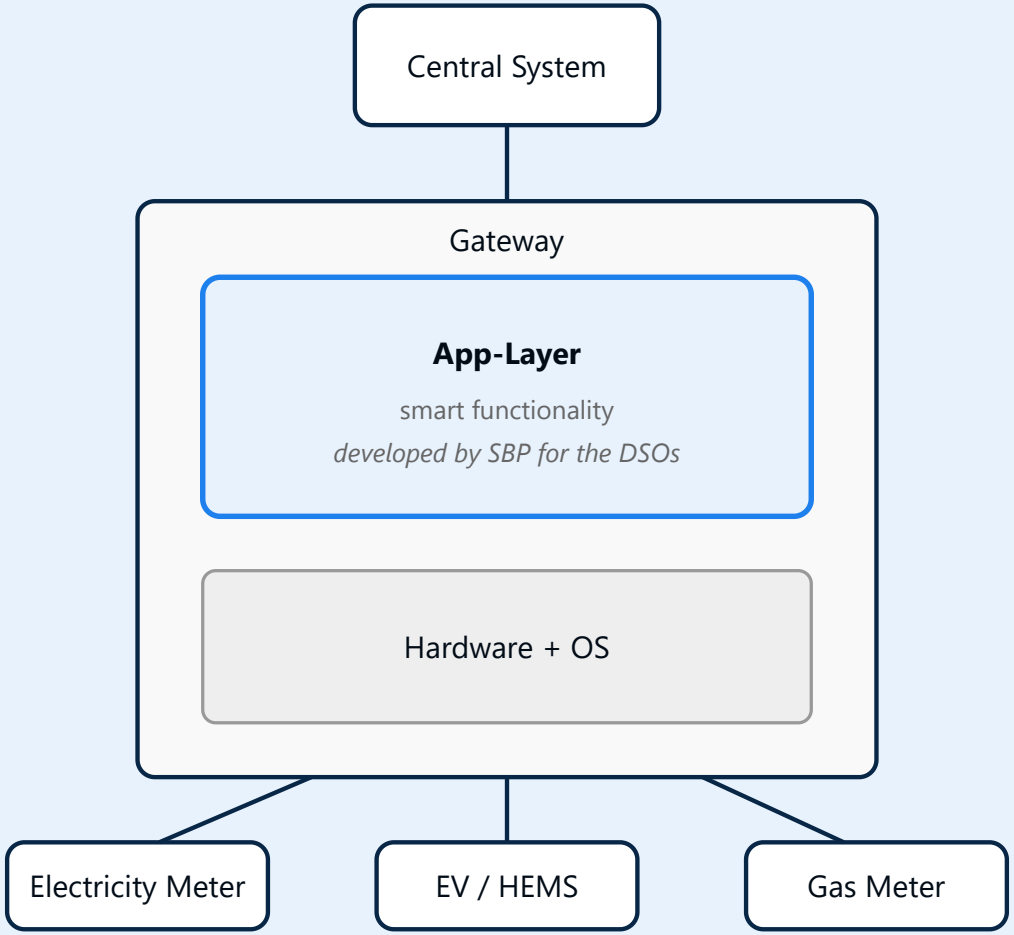
The DSMR6 Meter

- Electricity meter is no longer smart
- Smart functionality → separate gateway
- Gateway handles all communication
- Key insight: replace the gateway, not the meter



The App-Layer

- Procured separately from hardware and OS
- Ties everything together
- DSOs own the IP
- Dutch partner for flexibility — that's us



DSMR6 program goals

- **Cybersecurity**
  Security-by-design, privacy-by-design

DSMR6 program goals

- **Cybersecurity**
  Security-by-design, privacy-by-design
- **Minimal energy consumption**
  Efficient code, compact binaries

DSMR6 program goals

- **Cybersecurity**
  Security-by-design, privacy-by-design
- **Minimal energy consumption**
  Efficient code, compact binaries
- **Ownership transfer**
  DSOs must be able to maintain the software independently

DSMR6 program goals

- **Cybersecurity**
  Security-by-design, privacy-by-design
- **Minimal energy consumption**
  Efficient code, compact binaries
- **Ownership transfer**
  DSOs must be able to maintain the software independently
- **Industry momentum**
  Choose technologies that are growing, not declining

Why We Bid

- **We liked this project**

  The chance to run our software in all 8 million Dutch households

Why We Bid

- **We liked this project**

  The chance to run our software in all 8 million Dutch households

- **But only if we could build it in Rust**

  Let's walk through how we came to that conclusion

Why We Bid

- **We liked this project**

  The chance to run our software in all 8 million Dutch households

- **But only if we could build it in Rust**

  Let's walk through how we came to that conclusion

- **Bonus: The RFP included a hackathon**

  A lot of fun during the process

Language selection

C C++ Java Go C# Python Typescript **Rust**

These are the languages we use at Schuberg Philis.

Which one fits this job? We filtered based on program goals.

Filter 1 - Must Be Memory-Safe

~~C~~ ~~C++~~ Java Go C# Python Typescript **Rust**

- **~70% of critical vulnerabilities are memory safety bugs**
  Microsoft, Google data
- **NSA (2022): "avoid C and C++ for new development"**

Filter 2 - Must Run Without GC on Low Power

~~C~~ ~~C++~~ ~~Java~~ ~~Go~~ ~~C#~~ ~~Python~~ ~~Typescript~~ **Rust**

No garbage collector. No runtime. Low power budget.

Language	Energy vs Rust
Rust, C	baseline
Java, Go, C#	~2x
Python	~75x

(Pereira et al., 2017)

Filter 3 - Industry Momentum & Ownership

Rust ✓

- **DSOs will maintain this independently after handover**
  The technology must be growing, not niche
- **Rust: backed by Amazon/Google/Microsoft**
  In the Linux kernel, growing talent pool
- **Active community, modern tooling**
  `cargo`, crates.io

Risks We Identified

- **Rust is not the standard choice in metering**
  Industry defaults to C/C++ - evaluators may not know Rust

Risks We Identified

- **Rust is not the standard choice in metering**
  Industry defaults to C/C++ - evaluators may not know Rust
- **Quality tooling isn't mature for Rust**
  Hackathon scoring would use tools that don't fully support Rust

How We Mitigated Them

- **Signalled our intent to use Rust early**

  Gave the process time to understand and adapt

How We Mitigated Them

- **Signalled our intent to use Rust early**
  Gave the process time to understand and adapt
- **Raised the tooling gap as a concern**
  Scoring criteria were adjusted for a level playing field

How We Mitigated Them

- **Signalled our intent to use Rust early**
  Gave the process time to understand and adapt
- **Raised the tooling gap as a concern**
  Scoring criteria were adjusted for a level playing field
- **Lesson: don't surprise during procurement - be proactive**

Where Rust Helped

- **Software Development Lifecycle**

  Rust + mission-critical IT experience → secure, robust, future-proof

Where Rust Helped

- **Software Development Lifecycle**

  Rust + mission-critical IT experience → secure, robust, future-proof

- **Hackathon**

  Borrow checker + type system + crate ecosystem → full speed with confidence

Where Rust Helped

- **Software Development Lifecycle**

  Rust + mission-critical IT experience → secure, robust, future-proof

- **Hackathon**

  Borrow checker + type system + crate ecosystem → full speed with confidence

- **Energy efficiency**

  Compiled, no runtime, no GC

Result: We won the bid

SCHUBERG PHILIS

Challenge - Hiring

- **Rust talent is scarce**
  Dutch language requirement made it significantly harder

Challenge - Hiring

- **Rust talent is scarce**
  Dutch language requirement made it significantly harder
- **Turned down qualified candidates over (human) language, not skill**

Challenge - Hiring

- **Rust talent is scarce**
  Dutch language requirement made it significantly harder
- **Turned down qualified candidates over (human) language, not skill**
- **Easier to upskill internally**
  Senior SWEs from other teams already knew our way of working

Challenge - Hiring

- **Rust talent is scarce**
  Dutch language requirement made it significantly harder
- **Turned down qualified candidates over (human) language, not skill**
- **Easier to upskill internally**
  Senior SWEs from other teams already knew our way of working
- **Hired externally eventually - but it took a while**

Challenge - Ecosystem Gaps

- **No usable crates for key protocols**
  wM-Bus, DLMS, A-XDR encoding

Challenge - Ecosystem Gaps

- **No usable crates for key protocols**
  wM-Bus, DLMS, A-XDR encoding
- **Significant crypto work**
  OP-TEE and PKCS#11 interfaces

Challenge - Ecosystem Gaps

- **No usable crates for key protocols**
  wM-Bus, DLMS, A-XDR encoding
- **Significant crypto work**
  OP-TEE and PKCS#11 interfaces
- **When crates don't exist, you build them**
  Plan accordingly

Challenge - Quality Measurement Tooling

- **SLA on software quality**

  Measured by 3rd-party tooling

Challenge - Quality Measurement Tooling

- **SLA on software quality**

  Measured by 3rd-party tooling

- **Tooling is often immature for Rust**

  Close contact with vendors to improve measurements

Challenge - Quality Measurement Tooling

- **SLA on software quality**

  Measured by 3rd-party tooling

- **Tooling is often immature for Rust**

  Close contact with vendors to improve measurements

- **Scores can drop when measurements are tweaked**

  Part of the deal as an early adopter

Challenge - Logic Bugs Under Pressure

- **The borrow checker catches memory bugs, not logic bugs**

  Under hackathon pressure, you still make mistakes

Challenge - Logic Bugs Under Pressure

- **The borrow checker catches memory bugs, not logic bugs**

  Under hackathon pressure, you still make mistakes

- **Datetime conversions**

  Rust did not save me from some mistakes around datetimes

Challenge - Logic Bugs Under Pressure

- **The borrow checker catches memory bugs, not logic bugs**
  Under hackathon pressure, you still make mistakes
- **Datetime conversions**
  Rust did not save me from some mistakes around datetimes
- **Rust won't save you from yourself**
  But at least the bugs that slip through are the boring kind

What Went Well

- **High quality crates in the ecosystem**
  Example: A-XDR codec built on `rasn`, an ASN.1 serialization framework

What Went Well

- **High quality crates in the ecosystem**

  Example: A-XDR codec built on `rasn`, an ASN.1 serialization framework

- **Onboarding faster than expected**

  Regardless of background: C/C++, Java/C#, Python

What Went Well

- **High quality crates in the ecosystem**

  Example: A-XDR codec built on `rasn`, an ASN.1 serialization framework

- **Onboarding faster than expected**

  Regardless of background: C/C++, Java/C#, Python

- **PR feedback decreasing over time**

  Team matures, tech lead becomes redundant...

Takeaways

- **Rust is an excellent fit**

  For long-lived, security-sensitive, resource-constrained systems

Takeaways

- **Rust is an excellent fit**

  For long-lived, security-sensitive, resource-constrained systems

- **Communicate your language choice early**

  Give the process time to adapt

Takeaways

- **Rust is an excellent fit**

  For long-lived, security-sensitive, resource-constrained systems

- **Communicate your language choice early**

  Give the process time to adapt

- **Hiring is the bottleneck**

  Have a plan - start earlier than you think

Thank You

Ron van der Pols

Tech Lead - Schuberg Philis

SCHUBERG PHILIS

References

- Pereira et al., "Energy Efficiency across Programming Languages" (SLE 2017)
- NSA, "Software Memory Safety" (November 2022)
- Microsoft Security Response Center, "A proactive approach to more secure code" (2019)
- Chromium Project, "Memory safety" (ongoing)